

NotaLang

Un lenguaje para el cálculo de notas y promedios ponderados

Teoría de Compiladores 2026-2 — Trabajo Parcial (Hito 1)

Integrantes: [Nombre 1] · [Nombre 2] · [Nombre 3] · [Nombre 4]

Prof. José Luis Soncco Álvarez

Problemática y motivación

- Cada ciclo calculamos promedios ponderados y «¿cuánto necesito en el final?» a mano o en Excel.
- Errores frecuentes: pesos que no suman 100 %, notas fuera de 0–20, fórmulas mal copiadas.
- Excel no conoce el dominio: una nota de 25 es «un número válido».
- **NotaLang**: describir cursos de forma declarativa y que el **compilador detecte** esos errores.

Objetivos

- Definir el léxico y especificarlo con expresiones regulares en ANTLR4.
- Diseñar gramáticas libres de contexto no ambiguas y validarlas con derivaciones más a la izquierda.
- Implementar tabla de símbolos con ámbitos y un analizador semántico (Visitor).
- Verificar reglas del dominio: pesos = 100 %, notas en $[0, 20]$, evaluaciones existentes.

Un programa en NotaLang

```
curso Compiladores {  
    PC1 : 15% = 14;  
    TP  : 20% = 16.5;  
    PC2 : 15%;  
    TF  : 25%;  
    EB  : 25%;  
}  
var meta : decimal = 13;  
mostrar("Promedio parcial: ", promedio(Compiladores));  
mostrar("Necesito: ", necesito(Compiladores, meta));  
Compiladores.PC2 = 12;  
si (promedio(Compiladores) >= 10.5) { mostrar("Vas aprobando"); } sino { mostrar("Cuidado"); }
```

Construcciones del lenguaje

Construcción	Ejemplo
C1 Declaración	<code>var nota : decimal = 14.5;</code>
C2 Asignación	<code>Compiladores.PC1 = 15;</code>
C3 Expresiones	<code>promedio(C) < 11 o C.EF == 0</code>
C4 Selectiva	<code>si (p >= 10.5) { ... } sino si (p >= 8) { ... }</code>
C5 Iterativas	<code>mientras (i < 10) { ... } · para i desde 1 hasta n { ... }</code>
C6 Entrada / salida	<code>leer(x); · mostrar("Prom: ", promedio(C));</code>
C7 Curso (dominio)	<code>curso Calculo { PC : 40% = 12; EF : 60%; }</code>

Analizador léxico

Token	Lexemas / patrón
Palabras clave	var, curso, si, sino, mientras, para, desde, hasta, leer, mostrar, promedio, necesito
Tipos	entero, decimal, texto, logico
Operadores	+ - * / mod · < > <= >= == != · y o no · =
NUM_ENT / NUM_DEC	[0-9]+ · [0-9]+ ' . ' [0-9]+
PORCENTAJE	[0-9]+ (' . ' [0-9]+)? '%'
CADENA / ID	' ' ' ~["\r\n"]* ' ' ' · [a-zA-Z_][a-zA-Z_0-9]*
Ignorados	espacios, // . . . , /* . . . */
Cualquier otro carácter → ERROR_LEXICO con línea y columna.	

Analizador sintáctico (extracto de la GLC)

declCurso → **curso id { evaluaciones }**
evaluaciones → *evaluacion evaluaciones* | *evaluacion*
evaluacion → **id : porcentaje** *notaOpcional ;*
notaOpcional → **=** *expr* | ϵ

exprArit → *exprArit opSuma termino* | *termino*
termino → *termino opMul factor* | *factor*
siStmt → **si** (*expr*) *bloque sinoParte*
sinoParte → **sino** *bloque* | **sino** *siStmt* | ϵ

- Un no-terminal por nivel de precedencia; llaves obligatorias $\Rightarrow$ sin e/se colgante.

Derivación más a la izquierda (ejemplo)

```
curso Calculo { PC : 40% = 12; EF : 60%; }
```

declCurso

```
⇒ curso id { evaluaciones }  
⇒ curso id { evaluacion evaluaciones }  
⇒ curso id { id : porcentaje notaOpcional ; evaluaciones }  
⇒ curso id { id : porcentaje = expr ; evaluaciones }  
⇒* curso id { id : porcentaje = num_ent ; evaluaciones }  
⇒ curso id { id : porcentaje = num_ent ; evaluacion }  
⇒ curso id { id : porcentaje = num_ent ; id : porcentaje notaOpcional ; }  
⇒ curso id { id : porcentaje = num_ent ; id : porcentaje ; }
```

36 derivaciones (4 por gramática) en el Anexo A.

Analizador semántico

Cód	Error	Ejemplo
E1	Identificador no declarado	<code>b = a + 1;</code>
E2	Ya declarado en el ámbito / evaluación repetida	<code>var x : entero; var x : decimal;</code>
E3	Tipos incompatibles en asignación	<code>var edad : entero = 3.5;</code>
E4	Condición no lógica	<code>si (n) { ... }</code>
E5	Pesos del curso $\neq$ 100 %	<code>curso Q { PC1 : 30%; EF : 50%; }</code>
E6	Nota fuera de [0, 20]	<code>T1 : 50% = 25;</code>
E7	Evaluación inexistente / no es curso	<code>Ingles.ESCRITO = 15;</code>
E8	Operación con tipos incompatibles	<code>"a" * 2</code>

Visitor de ANTLR + tabla de símbolos con pila de ámbitos (bloques y para).

Pruebas

```
$ python3 main.py tests/err_sem5_pesos.nota
```

```
Análisis léxico y sintáctico: OK
```

```
=== ERRORES SEMÁNTICOS ===
```

```
[Error Semántico E5, línea 1] Los pesos del curso 'Quimica' suman 80% (deben sumar 100%)
```

```
$ python3 main.py tests/err_lex1.nota
```

```
=== ERRORES LÉXICOS ===
```

```
[Error Léxico, línea 1:19] carácter no reconocido '#'
```

- 16 archivos de prueba: 3 válidos, 2 léxicos, 3 sintácticos, 8 semánticos — todos con el resultado esperado.
- `make test` ejecuta todas las pruebas.

Conclusiones y siguiente hito

- Analizadores léxico, sintáctico y semántico funcionales, con 8 errores semánticos verificados.
- Las reglas del dominio (pesos, rango de notas) se detectan en compilación, no en ejecución.
- Siguiente: arquitectura del compilador, plan de validación y generación de código (LLVM) para ejecutar promedio y necesito.